

Practical 3

Implement the WaterJug Problem. The state of the system is represented as (x,y), where x is the quantity of water in the 4-gallon jug and y is the quantity of water in the 3-gallon jug. Assume the initial state to be (0,0) and the goal state to be (2,0). Output the sequence of steps to go from the initial to the goal state.

CODE :

```
#include<stdio.h>

int qx[100],qy[100];
int front=0, rear=0;
int visited[5][4];
int px[5][4] , py[5][4];
void printPath(int x, int y)
{
    if(x==-1 && y==-1)
    {
        return;
    }
    printPath(px[x][y],py[x][y]);
    printf("(%d,%d)\n",x,y);
}
void enqueue(int x, int y)
{
    qx[rear]=x;
    qy[rear]=y;
    rear++;
}
int main(){
    int x,y,nx,ny;
    enqueue(0,0);
```

```
visited[0][0]=1;
```

```
px[0][0]=-1, py[0][0]=-1;
```

```
while(front < rear)
```

```
{
```

```
    x=qx[front];
```

```
    y=qy[front];
```

```
    front++;
```

```
    if(x==2 && y==0)
```

```
    {
```

```
        printf("\nWater Jug solution using BFS approach will be:\n");
```

```
        printPath(x,y);
```

```
        return 0;
```

```
    }
```

```
    if(x<4 && !visited[4][y])
```

```
    {
```

```
        visited[4][y]=1;
```

```
        px[4][y]=x;
```

```
        py[4][y]=y;
```

```
        enqueue(4,y);
```

```
    }
```

```
    if(y<3 && !visited[x][3])
```

```
    {
```

```
        visited[x][3]=1;
```

```
        px[x][3]=x;
```

```
        py[x][3]=y;
```

```
    enqueue(x,3);  
}
```

```
if(x>0 && !visited[0][y])  
{  
    visited[0][y]=1;  
    px[0][y]=x;  
    py[0][y]=y;  
    enqueue(0,y);  
}
```

```
if(y>0 && !visited[x][0])  
{  
    visited[x][0]=1;  
    px[x][0]=x;  
    py[x][0]=y;  
    enqueue(x,0);  
}
```

```
if(y>0)  
{  
    int pour=(x+y<=4)?y:4-x;  
    nx=x+pour;  
    ny=y-pour;  
    if(!visited[nx][ny])  
    {  
        visited[nx][ny]=1;  
        px[nx][ny]=x;  
        py[nx][ny]=y;
```

```

        enqueue(nx,ny);
    }
}

if(x>0)
{
    int pour=(x+y<=3)?x:3-y;
    nx=x-pour;
    ny=y+pour;
    if(!visited[nx][ny])
    {
        visited[nx][ny]=1;
        px[nx][ny]=x;
        py[nx][ny]=y;
        enqueue(nx,ny);
    }
}
}

printf("\nNo solution found.");

return 0;
}

```

OUTPUT :

```

Water Jug solution using BFS approach will be:
(0,0)
(0,3)
(3,0)
(3,3)
(4,2)
(0,2)
(2,0)
PS E:\AIA> █

```